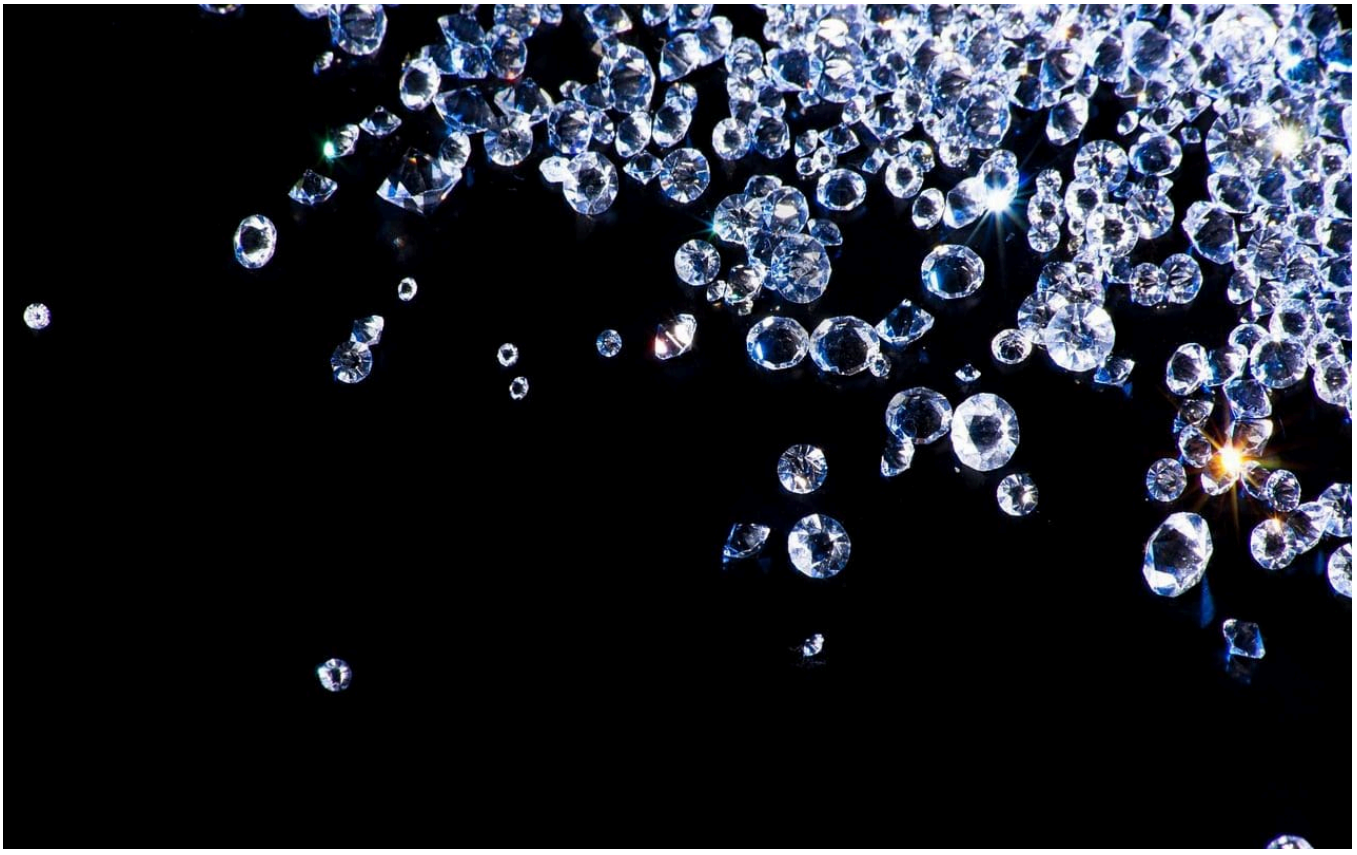
Synthetic Data Diamonds

mostly.ai/blog/synthetic-data-diamonds

Michael Platzer February 21, 2019



Synthetic diamonds are near indistinguishable from actual diamonds to the human eye, they have the same structure, and they bear they same valuable characteristics, like hardness, purity and thermal conductivity. Yet, they take 3 billion years less to form, come at a fraction of cost and are considered more ethical in terms of sourcing.

Along the same lines, synthetic data can as well be near *indistinguishable* from actual data, can have the *same structure*, and can retain all the same *valuable properties* (=statistical information) of actual data. Yet, machines can generate synthetic data in unlimited quantities, and more importantly, synthetic data allows big data to be utilized and shared without putting anyone's privacy at risk. Synthetic data has the potential to become the new risk-free & ethical norm to leverage customer data at scale. Finally, there is a solution for **big data privacy!**

However, just like with diamonds, the process of generating high quality synthetic data is anything but trivial. At MOSTLY AI we've mastered the automated generation of synthetic structured data over the past 2 years by leveraging state-of-the-art generative AI, and are confident to claim that we offer the world's most advanced synthetic data engine. As a simple demonstration, as part of this first post, let's apply our solution to a publicly available diamonds dataset with 53'940 records, and 10 attributes. As can be seen, the attributes are a mix of categorical as well as numerical variables.

	carat	cut	color	clarity	depth	table	price	x	y	z
1	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
2	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
3	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
4	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
5	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
6	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
7	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
8	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53

Fig. 1: First 8 records of the original diamonds dataset

Generating an equally sized, structural identical, yet synthetic version of this dataset is, thanks to the flexibility of our engine, as simple as:

> mostly train diamonds.csv

> mostly generate -n 53940

This will create a new data file with 53’940 records, where none of the generated records have any direct relationship anymore to the records in the original dataset. Hence, the information and thus privacy of any individual diamond is being protected (in case they care :)), while the structure of the population is retained.

	carat	cut	color	clarity	depth	table	price	x	y	z
1	0.32	Premium	I	SI1	61.5	58.0	508	4.43	4.39	2.72
2	2.07	Ideal	H	SI2	60.8	56.0	12920	8.20	8.29	5.01
3	0.31	Good	E	SI1	63.8	58.0	537	4.33	4.28	2.74
4	1.05	Very Good	G	VVS2	62.9	57.0	8173	6.45	6.42	4.05
5	0.45	Premium	J	VS1	60.7	60.0	898	4.99	4.94	3.01
6	0.90	Premium	H	VVS1	61.0	58.0	4931	6.23	6.27	3.84
7	1.10	Ideal	E	IF	62.9	55.0	12508	6.61	6.59	4.13
8	0.75	Ideal	E	VS2	61.1	56.0	3169	5.82	5.85	3.57

Fig. 2: First 8 records of synthetically generated diamonds dataset

Further the statistical properties of the various attributes are successfully retained. Here are side-by-side comparison of the frequency of the three categorical variables clarity, cut and color. All these univariate distributions are matched to near perfection.

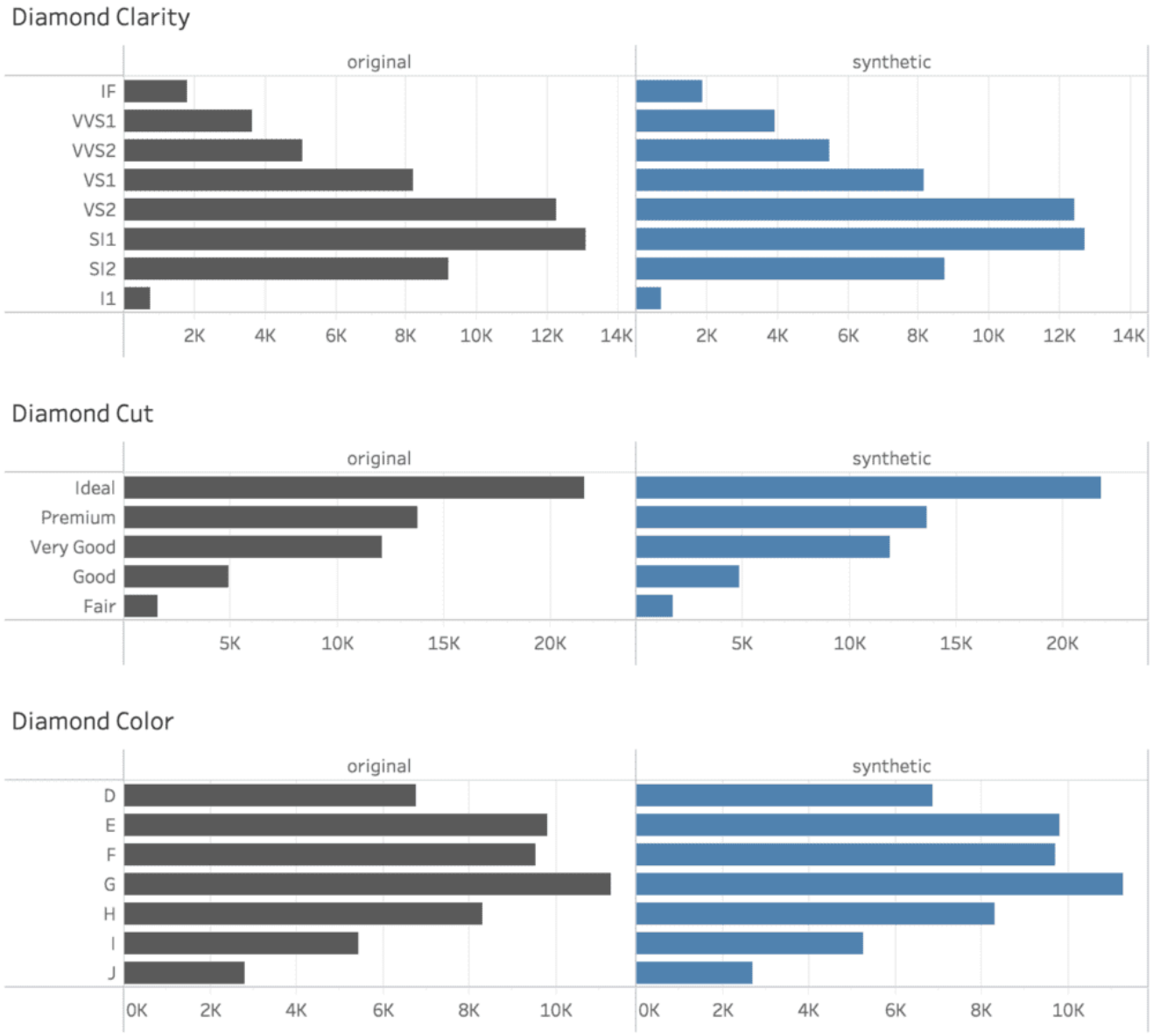


Fig. 3: Side-by-side comparison of distributions of categorical variables

Similarly, the engine is also able to retain the distributions of the numerical variables, and nicely captures location, skew, the tails as well as the multiple modes.

	q05	q10	q25	median	q75	q90	q95
original	$544	$646	$950	$2,401	$5,324	$9,821	$13,107
synthetic	$552	$666	$986	$2,388	$5,451	$9,841	$13,363

side-by-Fig. Fig. 4: Side comparison of price percentiles

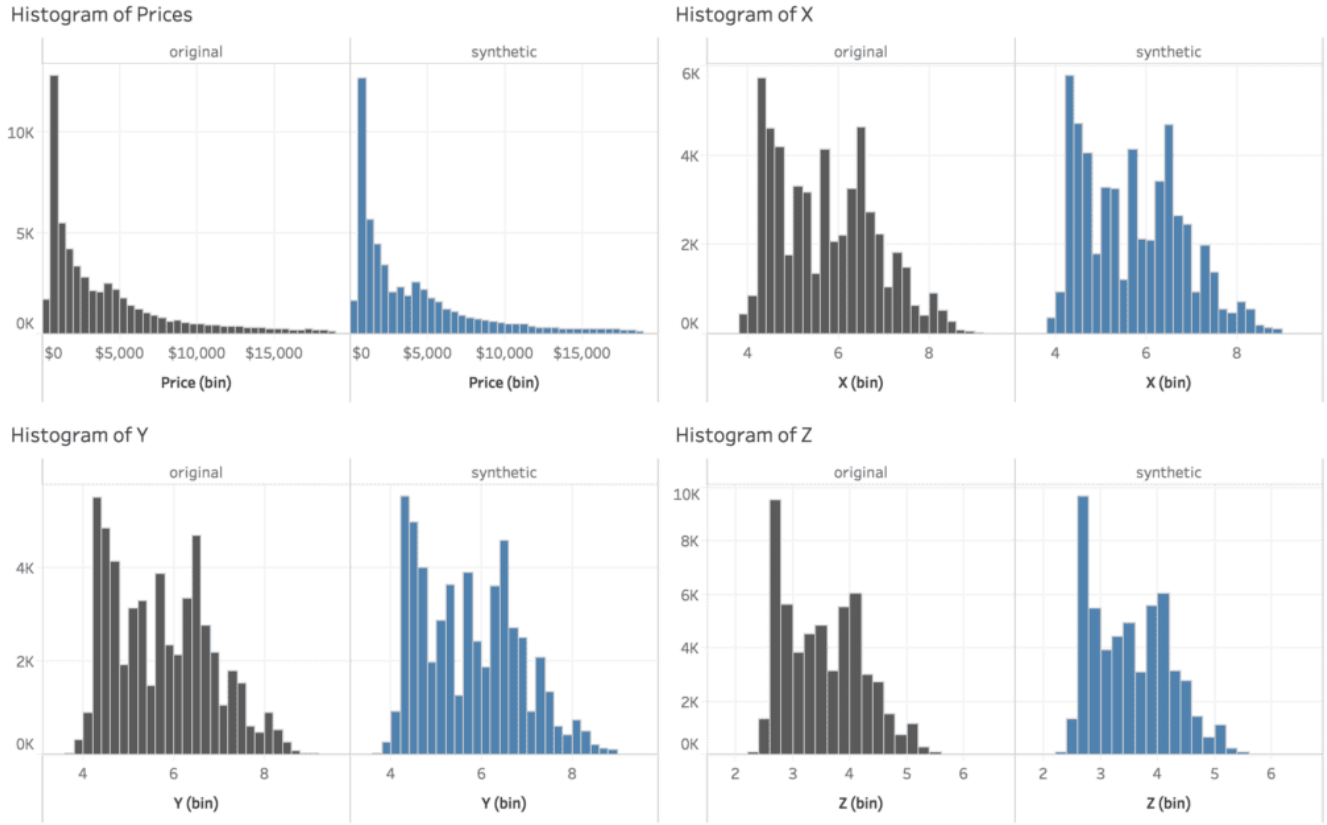


Fig. 5: Side-by-side comparison of histograms for price, x, y and z

More importantly, the (non-linear) interdependencies between the various variables is being properly captured. These statistical multivariate relationships are the key **value** of any dataset, as they help to gain a deeper understanding of the underlying domain.

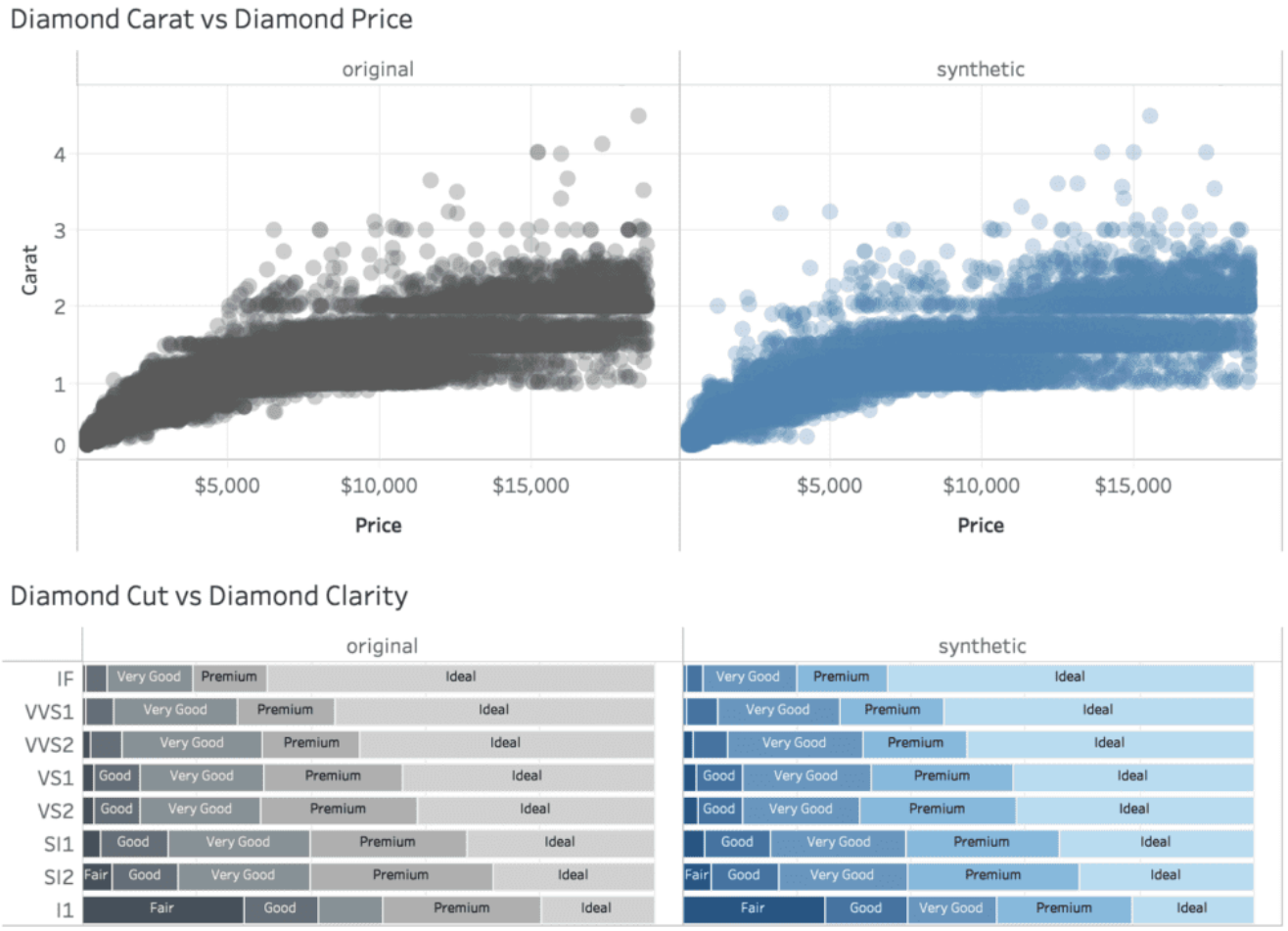


Fig. 6: Comparison of non-linear interdependencies between various variables

As can be seen, the patterns of the synthetic dataset mimic the patterns found in the original dataset to a very high degree. By analyzing the synthetic data we can gain various insights in terms of the diamond market. Whether that's the relationship between market prices and carats, or the dominance of "Ideal" cut for diamonds with clarity "IF". Even unexpected patterns, like the lack of "just-little-less-than-2-carat" diamonds is perfectly captured as well, if you take a close look at the upper chart.

Apart from descriptive statistics the synthetic data also turns out to be of immense value for training machine learning algorithms and predictive tasks. This opens up a whole range of opportunities to power next-generation AI, as more data can be gathered across different sources, across borders and industries, without infringing individual's privacy.

Let's now take a subset of 40k actual records, and generate 40k synthetic records based on that. We will need the remaining 13'940 actual records to have a fair holdout set available to benchmark our two models for diamond prices: one model trained on actual data, and another one on synthetic data. As expected, there is a loss in accuracy, as the data synthesis results in an information loss (to protect privacy), but the loss is within a rather small margin. In terms of mean absolute error we acchieve an error of 350$ vs 296$. Still, the 350$ is a far cry from a naive model, that would in this case result in an error of $3036 when having no data available at all.

```r

# read in data
act <- data.table::fread('diamonds/diamonds-40k-tgt.csv') #  40'000 actual records
syn <- data.table::fread('diamonds/diamonds-40k-syn.csv') #  40'000 synthetic records
val <- data.table::fread('diamonds/diamonds-40k-val.csv') #  13'940 actual holdout

# train predictive models for price based on all other attributes
m_act <- ranger::ranger(price ~ ., data = act, min.node.size = 25)
m_syn <- ranger::ranger(price ~ ., data = syn, min.node.size = 25)

# make predictions on a holdout dataset
val[, price_pred_act := predict(m_act, val)$predictions]
val[, price_pred_syn := predict(m_syn, val)$predictions]

# compare predictive accuracy in terms of mean absolute error
mae <- function(x, y) mean(abs(x-y))
mae(val$price, val$price_pred_act)
# 296 -> MAE based on actual training data
mae(val$price, val$price_pred_syn)
# 350 -> MAE based on synthetic training data

# compare predictive accuracy in terms of root mean squared logarithmic error
rmsle <- function(x, y) sqrt(mean(log(x/y)^2))
rmsle(val$price, val$price_pred_act)
# 0.104 -> RMSLE based on actual training data
rmsle(val$price, val$price_pred_syn)
# 0.118 -> RMSLE based on synthetic training data

```

Fig. 7: R code for benchmarking models trained on actual vs. synthetic data

In an upcoming post we will dive deeper into an example of synthetic *sequential* data, which actually turns out to be the prevalent form when it comes to privacy-sensitive user data. But this is a story for another day. For now, let's simply generate us a million more synthetic data diamonds and wallow in these for the time being.

> mostly generate -n 1000000 > mostly wallow 🙂

[Sign up for our Newsletter]